**Assignment: Data Fragmentation**
CSE 511: Data Processing at Scale

## Introduction

In this assignment, you will implement **data fragmentation** over a single base table in PostgreSQL using two strategies:

1. **Range partitioning** on a chosen numeric/timestamp column; and
2. **Round-robin partitioning** using table inheritance and a trigger.

You will also write a loader that builds a base table from a CSV file using a JSON header specification. The goal is to practice data modeling, loading, and partitioning mechanics while preserving correctness and distribution properties. Some more details on the same can be found here.

We will be using the `subreddits.csv` file and the `created_utc` column to create partitioned tables.

### Step 0: Environment Setup (0 pts)

**Option A**: Local Environment of PostgreSQL 14. Install the `psycopg2` lib (Python Library) by pip. All related files can be downloaded here. You only need to modify `fragmentation.py`.

**Option B**: Docker image here. PostgreSQL starts automatically when the container starts.

- **fragmentation.py** — your implementation.
- **headers.json** — schema spec used by `load_data` to `CREATE TABLE` (column → PostgreSQL type). Must match the CSV columns.
- **insert1.json / insert2.json / insert3.json** — small test rows used by the tester to verify inserts/trigger behavior (e.g., round-robin ongoing inserts).
- **subreddits.csv** — the dataset to load (large ~244 MB). Used by `load_data`.
- **tester.py** — the grading/test harness that imports `fragmentation.py`, runs your functions, and checks results.
- **test_helper.py** — helper utilities called by `tester.py` (table cleanup, assertions, sample inserts, etc.).

Test your code by: `python3 tester.py`

Copyright © Zhichao Cao, Ph.D.

### Step 1: Loading the data (20 pts)

To start you off, we have provided the code that creates a table for you. You **only** need to implement loading the data into the table provided.

**load_data(table_name, csv_path, connection, header_file)**

- Load csv_path using COPY in Python.

**Hints:**

1. We have already covered the COPY SQL command in assignment 0. This time you should need to learn how to run SQL commands in Python.

### Step 2: Range Partition (40 pts)

Your code should generate N horizontal fragments for the column `created_utc` of the `subreddits` table and store them in PostgreSQL. The algorithm should partition the table based on N uniform ranges of the attribute `created_utc`.

PostgreSQL has an inbuilt range partition method that we will be taking advantage of for this step. Here are some references on how to perform range partitioning in Postgres:

1. dbi Blog (dbi-services.com)
2. PostgreSQL: Documentation: Table Partitioning (Declarative Partitioning)

**range_partition(data_table_name, partition_table_name, num_partitions, header_file, column_to_partition, connection)**

1. Create parent partition_table_name with PARTITION BY RANGE(column_to_partition).
2. Compute min_val, max_val from data_table_name on column_to_partition.
3. Compute partition_gap = ceil((max_val - min_val) / num_partitions).
4. Create children partition_table_name0..N-1 with: FOR VALUES FROM (min_val + i*gap) TO (min_val + (i+1)*gap) (note: TO is exclusive).
5. INSERT INTO parent SELECT * FROM data_table_name; to route rows into children.

**Hints:**

1. Create a table that is about to be partitioned.
    a. You can refer code to create tables in the `load_data()` function and implement it here with the required modifications.
2. Get the min and max values of `created_utc`. Decide on the ranges.
    a. The range should be uniform. In case the range is coming out to be float, round it up to the next number. So, if the created_utc is within [1, 102], and the number

Copyright © Zhichao Cao, Ph.D.

of partitions is 5 - the range is (102 - 1)/5 = ceil(20.2) = 21, which will be considered as 21 - the ranges for each partition will be, [1, 22), [22, 43), [43, 64), [64, 85), [85, 106).

   b. Data inserted during testing will always be within the range of values currently in the table.

3. Create the partition tables
   a. In case the partition_table_name is "range_part", and there are 5 partitions, then the name of the parent table will be range_part with the names of the child (partitions) tables as range_part0, range_part1, range_part2, range_part3, and range_part4.
4. Insert data from the table created in Step 1 into these partitions now.

### Step 3: Round Robin Partition (40 pts)

Your code should generate N horizontal fragments for the column `created_utc` of the `subreddits` table and store them in PostgreSQL. The algorithm should partition the table based on **the round-robin approach**.

PostgreSQL does not have built-in round-robin partitioning. For this purpose, we will use Triggers in Postgres to simulate this. A detailed walkthrough of this is present at:

1. PostgreSQL: Documentation: Table Partitioning (Partition using Inheritance)

**round_robin_partition(data_table_name, partition_table_name, num_partitions, header_file, connection)**

- Create parent partition_table_name (unpartitioned) using header_file.
- Create children partition_table_name0..N-1 using INHERITS (parent).
- Initial load: First compute ROW_NUMBER() in a subquery or common table expression (CTE). Then, in an outer query, assign each row to child partition i when: (row_number - 1) % num_partitions = i. ROW_NUMBER() starts at 1, while the child partition suffix starts at 0. The resulting child-table row counts must differ by no more than one.
- Ongoing inserts: create a PL/pgSQL BEFORE INSERT trigger on the parent table. For every new row:
  - 1. Count the rows stored directly in each child table.
  - 2. Select the child with the smallest row count.
  - 3. If multiple children have the same smallest count, select the child with the smallest numeric suffix.
  - 4. Insert the complete NEW row into that child.
  - 5. Return NULL so that the row is not also stored in the parent table.

Copyright © Zhichao Cao, Ph.D.

- Note: Alternative implementations are allowed only if they produce the same observable behavior and pass the provided tester.

**Hints:**

The reference will cover all the necessary steps; however, we might not need some of them. Some high-level ideas to implement this step:

1. Create a table that is about to be partitioned.
    a. You can refer to the code used to create tables in the load_data() function and implement it here with the required modifications.
2. Create the partition tables
    a. In case partition_table_name is "rrobin_part", and the number of partitions is 3, then the name of the parent table will be "rrobin_part" and the child (partition) tables will be rrobin_part0, rrobin_part1, rrobin_part2.
3. Insert data from our step 1 table into these partitions.
    a. You can simply iterate over the data table rows and insert rows into one partition after the other.

## Submission Requirements & Guidelines

**Individual work:** This is an individual assignment; all work must be your own.

**What to submit on Canvas:** Submit **exactly one** Python file named `fragmentation.py`.

**Filename/extra files penalty:** Using a different filename or submitting additional files will incur a **30-point penalty**.

## Submission Policies

- Late submissions will *absolutely not* be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit than to submit late for no credit.
- Every student needs to *work independently* on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited, and plagiarism will directly lead to the failure of this course. We will use anti-plagiarism tools to detect violations of this policy.

Copyright © Zhichao Cao, Ph.D.

**Grading Rubric (100 pts)**

### A. Load Data (20 pts)

- **(20)** Bulk load via `COPY` in Python: **PASSED** — **{actual_rows}** rows loaded (matches CSV).

### B. Range Partition (40 pts)

- **(10)** Parent table creation: **PASSED** — `PARTITION BY RANGE` created.
- **(10)** Partition tables: **PASSED** — all **{num_partitions}** partitions created.
- **(10)** Data distribution: **PASSED** — all **{original_rows}** rows are correctly routed across the {num_partitions} child partitions. No rows are stored directly in the parent table.
- **(10)** New inserts (`insert1.json`, `insert2.json`, `insert3.json`): **PASSED** — each row routed to the correct range partition.

### C. Round-Robin Partition (40 pts)

- **(5)** Parent table creation: **PASSED** — inheritance parent created.
- **(5)** Round-robin tables: **PASSED** — all **{num_partitions}** partitions created.
- **(10)** Initial distribution: **PASSED** — even distribution achieved (max−min ≤ 1).
- **(10)** Trigger implementation: **PASSED** — function and `BEFORE INSERT` trigger on parent.
- **(10)** New inserts (`insert1.json`, `insert2.json`, `insert3.json`): **PASSED** — each row inserted into the current least-filled child.

Copyright © Zhichao Cao, Ph.D.

```
chang@chang-pc:~/docker-file/fragmentation$ python3 tester.py
================================================================================
ASSIGNMENT: DATA FRAGMENTATION - GRADING RUBRIC
================================================================================
Total Points: 100
Step 1: Data Loading (20 points)
Step 2: Range Partition (40 points)
Step 3: Round Robin Partition (40 points)
================================================================================
A database named "fragmentation" already exists

------------------------------------------------------------
TESTING STEP 1: DATA LOADING (20 points)
------------------------------------------------------------

------------------------------------------------------------
TESTING STEP 2: RANGE PARTITION (40 points)
------------------------------------------------------------

------------------------------------------------------------
TESTING STEP 3: ROUND ROBIN PARTITION (40 points)
------------------------------------------------------------

================================================================================
FINAL GRADING RESULTS
================================================================================

Load Data:
  Status: PASSED
  Points: 20/20
  Details:
    - (20) Bulk load via COPY in Python: PASSED — 914067 rows loaded (matches CSV).

Range Partition:
  Status: PASSED
  Points: 40/40
  Details:
    - (10) Parent table creation: PASSED — PARTITION BY RANGE created.
    - (10) Partition tables: PASSED — all 5 partitions created.
    - (10) Data distribution: PASSED — 914067 rows correctly routed across 5 partitions.
    - (10) New inserts (insert1.json, insert2.json, insert3.json): PASSED — each row routed to the correct range partition.

Round Robin Partition:
  Status: PASSED
  Points: 40/40
  Details:
    - (5) Parent table creation: PASSED — inheritance parent created.
    - (5) Round-robin tables: PASSED — all 5 partitions created.
    - (10) Initial distribution: PASSED — even distribution achieved (max–min ≤ 1).
    - (10) Trigger implementation: PASSED — function and BEFORE INSERT trigger on parent.
    - (10) New inserts (insert1.json, insert2.json, insert3.json): PASSED — each row inserted into the current least-filled child.

TOTAL SCORE: 100/100 points
================================================================================

Press d to Delete all tables? d
chang@chang-pc:~/docker-file/fragmentation$
```

Copyright © Zhichao Cao, Ph.D.